CBM — Configuration Baseline Module

CBM — Configuration Baseline Module

OOF® Origin Open Foundation®

Independent Methodological Authority

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Parent Standard: Software Configuration Standard (SCFS)

Category: Governance & Enforcement

Subcategory: Software Lifecycle Governance

Type: Core Module

Governed Space: Software Configuration Baseline Governance

Module ID: SLGA-SCFS-CBM-001

Origin ID: OOF.CM.SLGA.04.01.01

Version: 1.0

Status: Canonical · Open Standard

Effective Date: 12.10.2026

Compatibility: OOF® Methodology OS™ · Modular OOF® Architecture Ecosystem

GIE-Compatible: Yes

AI-Readable: Yes

Authority: OOF® — OriginOpen® Foundation

Protection: MIP® — Methodological Intellectual Property

Canonical Language: English (UCL™)

Canonical Definition

The Configuration Baseline Module (CBM) governs the identification,

establishment, authorization, composition, preservation, validation, and lifecycle continuity of software configuration baselines.

CBM establishes the methodological conditions under which a defined

software configuration can be recognized as an identifiable, bounded, authorized, evidence-supported reference state against which material configuration changes, build conditions, artifact relationships, and actual configuration states may be assessed.

It governs the relationship between a software configuration object, its

constituent configuration items, their material relationships, applicable configuration parameters, version references, environmental context, baseline authority, supporting evidence, and the conditions under which the baseline remains applicable.

CBM distinguishes configuration documentation from an established baseline,

baseline approval from baseline validity, baseline integrity from actual configuration conformity, and a baseline reference from the software configuration actually operating.

It applies to software configuration baselines established or maintained by

humans, organizations, automated configuration systems, AI coding assistants, autonomous software development agents, and software infrastructure mechanisms.

Its principal output is:

Governed Configuration Baseline State

Its foundational distinction is:

Documented Configuration ≠ Governed Configuration Baseline.

Operational Role

The operational role of CBM is:

To ensure that software configuration baselines remain identifiable, legitimately established, sufficiently composed, evidence-supported, temporally applicable, and reconstructable as authoritative configuration reference states without confusing baseline establishment with actual software configuration realization.

CBM provides the baseline-governance foundation of SCFS.

It establishes the reference state against which relevant configuration

changes, build relationships, artifact-version correspondence, environmental conditions, and configuration drift may be evaluated.

CBM does not independently execute builds, authorize every subsequent configuration change, validate the complete software product, or prove that a deployed environment conforms to its baseline.

It provides a governed configuration reference for competent downstream

configuration and lifecycle governance.

Operational Question

Under whose authority, within which software scope and environmental conditions, and on the basis of what evidence can a defined software configuration be established, preserved, and relied upon as a valid baseline for subsequent configuration governance?

Position in Architecture Governance Flow

The SLGA™ governance flow is:

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-cutting:

Software Lifecycle Control

CBM operates within:

Software Configuration Standard (SCFS)

The SCFS internal governance flow is:

Configuration Baseline → Build Management → Artifact Version Integrity →

Configuration Change Control → Build Reproducibility & Environment

CBM governs:

Configuration Baseline

It is the first Core Module of SCFS.

Its principal output is:

Governed Configuration Baseline State

This output supports:

BMM — Build Management Module

AVIM — Artifact Version Integrity Module

CCCM — Configuration Change Control Module

BREM — Build Reproducibility & Environment Module

and contributes to the parent SCFS output:

Governed Software Configuration State

CBM may be invoked again whenever material baseline scope, composition, authority, version relationships, environmental assumptions, or applicability conditions change.

• Governed Space

Software Configuration Baseline Governance

This space governs the conditions under which software configuration

baselines are established and maintained as identifiable reference states.

Its principal dimensions include:

• Baseline governance object identity

• Dependent software object

• Baseline purpose

• Baseline scope

• Baseline classification

• Configuration item identification

• Configuration item relationships

• Baseline composition

• Component version references

• Artifact references

• Configuration parameter references

• Default configuration references

• Configuration override references

• Environmental assumptions

• Configuration constraints

• Baseline establishment authority

• Baseline approval conditions

• Baseline effective conditions

• Baseline evidence

• Baseline evidence competence

• Baseline evidence integrity

• Baseline validation

• Baseline completeness

• Baseline consistency

• Baseline uncertainty

• Baseline conflicts

• Baseline gaps

• Baseline change references

• Baseline supersession

• Baseline reconstruction

• Baseline drift references

• Baseline continuity

• Baseline governance states

The governed space is independent of any particular repository, configuration file format, build platform, software supplier, or deployment technology.

Configuration Baseline Governance Object

Every material CBM assessment shall establish a Configuration Baseline

Governance Object.

The object represents a defined software configuration reference and the

conditions under which that reference may be recognized as governed.

It should identify:

Baseline Identity: The identifiable configuration reference.

Software Object: The software system, service, application, or artifact

concerned.

Baseline Purpose: The function of the baseline.

Baseline Scope: The configuration items and relationships included.

Baseline Classification: The type or intended lifecycle role of the

baseline.

Configuration Items: The material constituent elements.

Composition Relationships: The relationships among included items.

Version References: Relevant software and component versions.

Parameter References: Relevant configuration settings.

Environment Context: The environment for which the baseline applies.

Authority Reference: The basis for baseline establishment.

Effective Conditions: The conditions under which the baseline is applicable.

Baseline Evidence: Records supporting the baseline.

Baseline Governance State: The recognized condition of the baseline.

The governance object shall remain distinguishable from a configuration

file, repository commit, build manifest, or deployed software instance.

Configuration Baseline Governance Object ≠ Configuration File.

Baseline Purpose Governance

CBM shall identify the purpose for which a software configuration baseline

is established.

Purposes may include:

• Development reference

• Integration reference

• Testing reference

• Release preparation

• Production configuration reference

• Maintenance reference

• Recovery reference

• Retirement reference

The purpose shall remain associated with the baseline's intended lifecycle

context.

Baseline Purpose ≠ Universal Baseline Applicability.

Baseline Scope Governance

CBM shall establish the boundaries of the baseline.

Scope may include:

• Software components

• Component versions

• Artifact references

• Configuration parameters

• Environment-specific settings

• Relevant defaults

• Required configuration relationships

• Material external references

Scope shall distinguish included, excluded, and unresolved configuration

elements.

Defined Baseline Scope ≠ Complete Baseline Composition.

Baseline Classification

CBM shall recognize the intended classification of a baseline.

Relevant classifications may include:

• Development baseline

• Integration baseline

• Test baseline

• Release candidate baseline

• Production reference baseline

• Maintenance baseline

• Recovery baseline

• Historical baseline

Classification shall not independently establish baseline approval or

validity.

Baseline Classification ≠ Baseline Governance State.

Configuration Item Identification

CBM shall identify material configuration items included within the baseline.

Items may include:

• Software components

• Executable artifacts

• Libraries

• Configuration files

• Parameter sets

• Runtime settings

• Build configuration references

• Interface settings

• External service configuration references

Each material item shall remain sufficiently identifiable within the baseline.

Configuration Item Listing ≠ Verified Configuration Item Identity.

Configuration Item Boundary

CBM shall establish the boundaries of material configuration items.

A configuration item may represent:

• An individual component

• A configuration file

• A parameter group

• An artifact reference

• A versioned configuration object

• A defined subsystem configuration

Item boundaries shall support reconstruction of baseline composition.

Configuration Item Name ≠ Configuration Item Boundary.

Configuration Item Relationships

CBM shall identify material relationships between baseline configuration

items.

Relationships may include:

• Component-to-version

• Artifact-to-configuration

• Parameter-to-function

• Runtime-to-platform

• Interface-to-service

• Configuration-to-environment

• Build-reference-to-artifact

Relationships shall be sufficiently explicit to support baseline

interpretation.

Configuration Item Presence ≠ Established Configuration Relationship.

Baseline Composition

CBM shall establish how material configuration items combine into the

baseline.

Composition shall preserve:

• Included items

• Relevant relationships

• Applicable version references

• Parameter references

• Environmental context

• Material constraints

A baseline may contain individually identifiable items while remaining compositionally incomplete or inconsistent.

Configuration Item Collection ≠ Governed Baseline Composition.

Baseline Version References

CBM shall identify material version references included in the baseline.

Version references may concern:

• Software application versions

• Component versions

• Runtime versions

• Artifact versions

• Configuration schema versions

• Relevant platform versions

Version references shall remain sufficiently precise for competent

downstream interpretation.

Version Label Presence ≠ Verified Version Correspondence.

Baseline Artifact References

CBM shall identify material software artifact references included in the

baseline.

References may concern:

• Executables

• Libraries

• Packages

• Containers

• Runtime images

• Generated artifacts

• Deployable bundles

CBM establishes the reference relationship.

Artifact-version integrity validation remains governed by AVIM.

Artifact Reference ≠ Verified Artifact Identity.

Baseline Parameter References

CBM shall identify material configuration parameters included in the

baseline.

Parameter references may include:

• Parameter identity

• Intended value

• Applicable range

• Configuration source

• Relevant default

• Override relationship

• Environmental applicability

Parameter inclusion shall not independently establish that the value is effective in an actual environment.

Baseline Parameter Value ≠ Effective Runtime Parameter Value.

Default Configuration References

CBM shall recognize material default settings upon which the baseline

depends.

Defaults may originate from:

• Software frameworks

• Libraries

• Runtime environments

• Configuration schemas

• Build tools

• External software services

A baseline that relies on material defaults shall preserve sufficient references to identify them.

Implicit Default ≠ Governed Baseline Value.

Configuration Override References

CBM shall identify material override mechanisms relevant to the baseline.

Overrides may arise through:

• Environment variables

• Runtime parameters

• Configuration services

• Deployment settings

• Command-line arguments

• Automated configuration mechanisms

CBM shall distinguish the baseline reference from the effective

configuration after overrides.

Baseline Reference ≠ Effective Configuration After Overrides.

Baseline Environmental Context

CBM shall establish the environmental context in which the baseline is intended to apply.

Relevant contexts may include:

• Development

• Testing

• Staging

• Production

• Maintenance

• Recovery

A baseline applicable to one environment shall not automatically be

considered valid for another.

Test Baseline Validity ≠ Production Baseline Validity.

Baseline Environmental Assumptions

CBM shall identify material environmental assumptions supporting the

baseline.

Assumptions may concern:

• Runtime capabilities

• Platform versions

• Available services

• Resource conditions

• Configuration mechanisms

• Required interfaces

• External dependencies

Unsupported assumptions shall remain explicit.

Assumed Environment Conditions ≠ Verified Environment Conditions.

Baseline Constraints

CBM shall preserve material constraints applicable to the baseline.

Constraints may originate from:

• Software requirements

• Dependency compatibility

• Platform conditions

• Configuration rules

• Security requirements

• Quality requirements

• Operational conditions

CBM governs their representation within the baseline.

Competent assessment of the originating requirements remains within the relevant governance domain.

Constraint Reference ≠ Verified Constraint Satisfaction.

Baseline Establishment Authority

CBM shall identify the legitimate basis for establishing a configuration

baseline.

Authority may arise through:

• Competent organizational approval

• Assigned configuration responsibility

• Delegated development authority

• Defined lifecycle control

• Applicable change-management decisions

Authority shall remain bounded by the relevant software object, baseline

scope, environment, and purpose.

Ability to Establish a Baseline ≠ Authority to Establish a Baseline.

Baseline Authority Scope

CBM shall assess whether baseline establishment falls within the actor's authorized scope.

Relevant dimensions may include:

• Actor identity

• Delegated responsibility

• Software object

• Baseline classification

• Configuration items

• Environment

• Permitted actions

• Applicable restrictions

Authority to establish a development baseline does not automatically establish authority to approve a production baseline.

Development Baseline Authority ≠ Production Baseline Authority.

Automated Baseline Establishment

CBM shall govern baseline establishment performed through automated systems.

Automated mechanisms may include:

• Build pipelines

• Configuration management systems

• Repository automation

• AI coding assistants

• Autonomous software development agents

Automation shall remain subject to applicable authority, scope, and

evidence requirements.

Automated Baseline Generation ≠ Authorized Baseline Establishment.

Baseline Approval

CBM shall preserve the relationship between a proposed baseline and its

competent approval.

Approval should identify:

• Approved baseline object

• Applicable authority

• Approved scope

• Relevant conditions

• Approval evidence

• Temporal applicability

Approval shall remain distinguishable from technical validation.

Approved Baseline ≠ Technically Validated Baseline.

Baseline Effective Conditions

CBM shall identify the conditions under which an established baseline

becomes applicable.

Conditions may include:

• Approval completion

• Required validation

• Applicable environment

• Relevant software version

• Lifecycle milestone

• Defined effective date

• Supersession of a previous baseline

An approved baseline may remain inapplicable until its effective conditions are satisfied.

Baseline Approval ≠ Immediate Baseline Applicability.

Baseline Temporal Applicability

CBM shall preserve the temporal conditions of baseline applicability.

Relevant information may include:

• Establishment time

• Approval time

• Effective time

• Supersession time

• Retirement time

• Relevant historical period

A baseline may remain historically valid without remaining currently applicable.

Historical Baseline Validity ≠ Current Baseline Applicability.

Baseline Identification Integrity

CBM shall preserve the integrity of baseline identity.

Relevant considerations may include:

• Unique baseline reference

• Stable identification

• Version or revision reference

• Constituent composition

• Evidence association

• Supersession relationships

Baseline identity shall not depend solely on an editable filename or

informal description.

Baseline Filename ≠ Reliable Baseline Identity.

Baseline Record Integrity

CBM shall preserve the integrity of material baseline records.

Relevant considerations may include:

• Record source

• Content integrity

• Modification history

• Approval evidence

• Temporal relevance

• Constituent item correspondence

• Preservation history

An intact baseline record does not independently establish that the baseline is complete or technically appropriate.

Baseline Record Integrity ≠ Baseline Validity.

Baseline Completeness

CBM shall assess whether the baseline sufficiently represents the material

configuration elements required within its defined scope.

Completeness may concern:

• Required configuration items

• Required version references

• Relevant parameter values

• Applicable defaults

• Material overrides

• Environmental assumptions

• Required relationships

Completeness shall be assessed relative to baseline purpose and scope.

Complete Baseline Record ≠ Complete Software Configuration Knowledge.

Baseline Consistency

CBM shall assess material consistency among baseline elements.

Relevant conditions may include:

• Version relationship consistency

• Parameter consistency

• Artifact reference consistency

• Environment compatibility assumptions

• Configuration relationship coherence

• Applicable constraint relationships

A baseline may be complete in documentation while internally inconsistent.

Baseline Completeness ≠ Baseline Consistency.

Baseline Validation

CBM shall determine whether material baseline conditions are sufficiently supported within the assessed scope.

Validation may consider:

• Baseline identity

• Purpose

• Scope

• Composition

• Version references

• Parameter references

• Environmental assumptions

• Authority

• Approval conditions

• Evidence competence

• Material uncertainty

Baseline validation shall remain distinguishable from validation of the

software system's actual runtime configuration.

Validated Baseline ≠ Verified Actual Configuration.

Baseline Evidence

CBM shall preserve evidence supporting material baseline claims.

Evidence may include:

• Configuration manifests

• Repository records

• Baseline specifications

• Approval records

• Version references

• Parameter records

• Artifact references

• Configuration snapshots

• Relevant change records

Evidence shall remain associated with the specific baseline claim it

supports.

Baseline Evidence Presence ≠ Baseline Claim Validity.

Baseline Evidence Competence

CBM shall assess whether available evidence is competent to support the relevant baseline determination.

For example:

A configuration manifest may support baseline composition.

An approval record may support baseline authority.

A version reference may support declared version identity.

An artifact inspection may support observed artifact correspondence.

These records establish different facts.

Evidence of Baseline Approval ≠ Evidence of Baseline Technical Validity.

Baseline Evidence Integrity

CBM shall preserve the integrity of material baseline evidence.

Relevant considerations may include:

• Evidence source

• Content integrity

• Temporal relevance

• Baseline correspondence

• Software object correspondence

• Material contradictions

• Preservation history

Evidence integrity shall not independently establish baseline correctness.

Intact Baseline Evidence ≠ Correct Baseline Composition.

Baseline Uncertainty

CBM shall preserve material uncertainty concerning baseline identity,

authority, composition, applicability, or evidence.

Uncertainty may arise where:

• Baseline scope is incomplete

• Configuration items remain unidentified

• Version references are ambiguous

• Approval authority is unresolved

• Environmental assumptions are unsupported

• Effective conditions are unclear

• Material evidence is missing

Uncertainty shall not be silently converted into baseline acceptance.

Unresolved Baseline Conditions ≠ Verified Baseline Conditions.

Baseline Conflict

A Baseline Conflict exists where material baseline claims, configuration references, authority records, or evidence are incompatible.

Examples may include:

• Conflicting approved baseline versions

• Contradictory parameter references

• Incompatible configuration item relationships

• Conflicting approval records

• Inconsistent effective dates

• Contradictory environmental applicability claims

CBM shall preserve the conflict and its supporting evidence.

Conflicting Baseline Records ≠ Permission to Select the Preferred

Baseline.

Baseline Gap

A Baseline Gap exists where material information required for competent baseline assessment remains insufficiently established.

Examples may include:

• Missing configuration item references

• Unknown version relationships

• Incomplete baseline composition

• Missing approval evidence

• Unresolved environment assumptions

• Missing effective conditions

CBM shall identify the gap and its material significance.

Baseline Evidence Gap ≠ Proven Baseline Failure.

Critical Baseline Gap

A Critical Baseline Gap exists where missing baseline information prevents competent assessment of a materially consequential software configuration reference.

Examples may include:

- A production baseline without sufficiently established constituent

artifact versions. - A critical software baseline without competent approval evidence. - A recovery baseline whose required configuration elements cannot be reconstructed. - A release preparation baseline with unresolved material configuration relationships.

CBM shall preserve the critical gap for competent downstream assessment.

It does not independently determine software shutdown, security compromise, or legal liability.

Critical Baseline Gap ≠ Automatic Software Failure.

Baseline Change Recognition

CBM shall identify material changes affecting an established baseline.

Changes may include:

• Configuration item addition

• Configuration item removal

• Version reference modification

• Parameter modification

• Environmental scope change

• Baseline authority change

• Effective condition change

CBM governs the baseline identity and reference implications of such

changes.

The detailed authority and execution of configuration changes are governed by CCCM.

Baseline Change Recognition ≠ Configuration Change Authorization.

Baseline Revision

CBM shall preserve the relationship between an existing baseline and a

proposed or established revision.

A revision should identify:

• Previous baseline

• Revised baseline

• Material differences

• Revision purpose

• Applicable authority

• Relevant evidence

• Effective conditions

A revised baseline shall remain distinguishable from an undocumented

modification of the previous baseline.

Baseline Revision ≠ Silent Baseline Replacement.

Baseline Supersession

CBM shall identify when one baseline supersedes another.

Supersession shall preserve:

• Superseded baseline identity

• Successor baseline identity

• Supersession basis

• Relevant authority

• Effective conditions

• Material historical references

Supersession shall not erase the historical baseline record.

Baseline Supersession ≠ Historical Baseline Erasure.

Baseline Retirement

CBM shall recognize when a baseline is no longer applicable as an active configuration reference.

Retirement may occur because of:

• Software replacement

• Configuration redesign

• Platform migration

• Lifecycle transition

• Supersession

• Software retirement

Baseline retirement shall preserve material historical identity and

evidence.

Retired Baseline ≠ Invalid Historical Baseline.

Baseline Reconstruction

CBM shall support reconstruction of material baseline states where sufficient evidence exists.

Reconstruction may use:

• Baseline specifications

• Configuration manifests

• Version references

• Artifact references

• Parameter records

• Approval records

• Historical snapshots

• Relevant change records

Reconstructed baseline states shall distinguish established facts from

assumptions.

Reconstructed Baseline ≠ Directly Observed Historical Configuration.

Baseline Drift Reference

CBM shall preserve references to material divergence between an applicable

baseline and observed configuration states.

CBM establishes the baseline reference against which drift may be evaluated.

Detailed configuration change control remains within CCCM.

Actual configuration realization and broader configuration validation

remain within SCFS and competent lifecycle governance.

Baseline Difference ≠ Automatically Unauthorized Configuration Drift.

Baseline Continuity

CBM shall preserve material baseline identity and relationships across

software lifecycle transitions.

Continuity may involve:

• Requirements references

• Creation references

• Dependency references

• Build references

• Artifact-version references

• Configuration change references

• Release references

• Deployment references

• Maintenance references

• Retirement references

Baseline continuity shall preserve the distinction between historical,

current, proposed, and superseded baseline states.

Baseline Continuity ≠ Baseline Immutability.

Baseline Reassessment

CBM shall support reassessment when material baseline conditions change.

Triggers may include:

• Configuration scope changes

• Version reference changes

• Artifact changes

• Environmental changes

• New baseline authority conditions

• Material evidence conflicts

• Critical baseline gaps

• Baseline supersession

• Changed lifecycle requirements

Reassessment shall preserve relevant historical baseline records.

Previously Valid Baseline ≠ Permanently Applicable Baseline.

Baseline Governance States

CBM shall support identifiable governance states.

Possible states include:

Identified: A baseline governance object has been recognized.

Proposed: A baseline has been specified but not established.

Under Assessment: Material baseline conditions are being evaluated.

Authority-Unresolved: Applicable baseline establishment authority remains

uncertain.

Composition-Unresolved: Material baseline composition remains

insufficiently established.

Environment-Unresolved: Relevant environmental applicability remains

uncertain.

Evidence-Limited: Available evidence is insufficient for the required determination.

Conditionally Established: Baseline establishment depends on explicit

conditions.

Established: A baseline has been legitimately established within its defined scope.

Validated: Material baseline conditions are sufficiently supported within the assessed scope.

Conflicted: Material baseline claims or evidence are contradictory.

Invalid: The baseline does not satisfy material requirements within the assessed scope.

Superseded: A later baseline has replaced the earlier baseline.

Retired: The baseline is no longer an active applicable reference.

Unresolved: Available information does not support a sufficiently determined state.

These states shall not independently establish actual runtime

configuration, software quality, security, release readiness, or deployment authorization.

Established Baseline ≠ Verified Operational Configuration.

Governed Configuration Baseline State

The principal output of CBM is:

Governed Configuration Baseline State

The state shall preserve sufficient information to identify:

Baseline Governance Object: The baseline assessment.

Baseline Identity: The identifiable configuration reference.

Software Object: The software system or artifact concerned.

Baseline Purpose: The intended baseline function.

Baseline Scope: The boundaries of the configuration reference.

Baseline Classification: The intended lifecycle role.

Configuration Items: The material constituent elements.

Configuration Item Boundaries: The recognized item boundaries.

Configuration Relationships: The relationships among included items.

Baseline Composition: The established configuration reference composition.

Version References: Relevant software and component versions.

Artifact References: Material software artifact references.

Parameter References: Material configuration parameters.

Default References: Relevant default configuration conditions.

Override References: Relevant configuration override mechanisms.

Environmental Context: The environment for which the baseline applies.

Environmental Assumptions: Material assumptions supporting applicability.

Baseline Constraints: Applicable limits and conditions.

Establishment Authority: The legitimate basis for baseline establishment.

Authority Scope: The boundaries of baseline authority.

Automation Reference: Relevant automated baseline establishment

mechanisms.

Approval Condition: The competent approval state.

Effective Conditions: Conditions required for baseline applicability.

Temporal Applicability: The relevant effective period.

Identification Integrity: The condition of baseline identity.

Record Integrity: The integrity of baseline records.

Baseline Completeness: The assessed completeness within scope.

Baseline Consistency: The assessed internal coherence.

Baseline Validation: The scope and result of baseline assessment.

Baseline Evidence: Evidence supporting material claims.

Evidence Competence: The suitability of evidence for those claims.

Evidence Integrity: The condition of supporting evidence.

Baseline Uncertainty: Material unresolved conditions.

Baseline Conflicts: Contradictory baseline claims or evidence.

Baseline Gaps: Missing material baseline information.

Critical Baseline Gaps: Material gaps requiring competent attention.

Baseline Changes: Material changes affecting the baseline.

Baseline Revisions: Relevant revision relationships.

Baseline Supersession: Successor and predecessor relationships.

Baseline Retirement: Relevant retirement conditions.

Baseline Reconstruction: Historical reconstruction information.

Baseline Drift References: Material divergence references.

Baseline Continuity: Preservation of material baseline relationships.

Baseline Reassessment: Relevant reassessment history.

Baseline Governance State: The recognized baseline condition.

Downstream References: Information required by competent SCFS and SLGA™

governance.

The state shall not independently establish actual software configuration,

build reproducibility, artifact integrity, release readiness, or deployment authorization.

Minimum Implementation Framework

A conforming implementation of CBM must provide the following six functional capabilities.

• Configuration Baseline Object, Purpose, Scope, and Classification

Governance

Capability to establish identifiable baseline governance objects, associate

them with the relevant software object, define baseline purpose, determine scope, and recognize the baseline's intended lifecycle classification.

The implementation must distinguish baseline documentation from an established configuration reference.

• Configuration Item, Composition, Version, Parameter, and Environment

Governance

Capability to identify material configuration items, establish their

boundaries and relationships, preserve baseline composition, and associate relevant version references, artifact references, parameter values, defaults, overrides, environmental conditions, and constraints.

The implementation must distinguish configuration item listing from sufficient baseline composition.

• Baseline Authority, Approval, Effective Conditions, and Temporal

Applicability Governance

Capability to establish the legitimate authority for baseline creation,

assess authority scope, preserve competent approval evidence, and identify effective and temporal applicability conditions.

The implementation must distinguish baseline approval from technical validation and current applicability.

• Baseline Evidence, Integrity, Completeness, Consistency, and Validation

Governance

Capability to associate baseline claims with competent evidence, preserve

record integrity, assess completeness and consistency, and determine whether material baseline conditions are sufficiently supported.

The implementation must distinguish an intact baseline record from a valid baseline and from actual software configuration conformity.

• Baseline Gap, Conflict, Revision, Supersession, Reconstruction, and

Continuity Governance

Capability to identify material baseline uncertainty, gaps, critical gaps,

conflicting evidence, revisions, supersession relationships, retirement conditions, reconstruction requirements, and continuity across lifecycle transitions.

The implementation must preserve historical baseline identity and evidence.

• Governed Configuration Baseline State and SCFS Handoff Governance

Capability to produce a Governed Configuration Baseline State with

sufficient identity, purpose, scope, composition, authority, applicability, evidence, validation, uncertainty, and continuity information for competent downstream SCFS governance.

The implementation must not represent baseline establishment as proof of actual runtime configuration or software product correctness.

GIE™ Compatibility

CBM is compatible with GIE™ as a downstream methodological analysis and execution layer.

GIE™ may support:

• Baseline governance object identification

• Baseline purpose analysis

• Baseline scope analysis

• Baseline classification

• Configuration item identification

• Configuration item boundary analysis

• Configuration relationship mapping

• Baseline composition assessment

• Version reference analysis

• Artifact reference analysis

• Parameter reference analysis

• Default configuration recognition

• Override reference identification

• Environmental context assessment

• Environmental assumption analysis

• Baseline constraint recognition

• Baseline authority assessment

• Authority scope analysis

• Automated baseline establishment analysis

• Baseline approval evidence association

• Effective condition assessment

• Temporal applicability analysis

• Baseline identification integrity assessment

• Baseline record integrity assessment

• Baseline completeness analysis

• Baseline consistency analysis

• Baseline validation support

• Baseline evidence competence assessment

• Baseline uncertainty recognition

• Baseline conflict detection

• Baseline gap detection

• Critical baseline gap detection

• Baseline revision analysis

• Baseline supersession mapping

• Baseline retirement recognition

• Baseline reconstruction support

• Baseline drift reference analysis

• Baseline continuity mapping

• Baseline reassessment support

• Governed baseline state representation

GIE™ may identify candidate baseline relationships, infer possible inconsistencies, or propose baseline assessments.

Such outputs shall remain distinguishable from sufficiently evidenced

baseline determinations.

GIE™ must not fabricate baseline approvals, invent configuration values, infer authority from technical access, or represent declared configuration as observed operational reality.

OOF® logic adapts to the governed reality. The engine adapts to OOF®

logic.

Governance Logic

The canonical CBM governance logic is:

Receive Governed Software Requirements Context

↓

Receive Governed Software Creation Context

↓

Receive Governed Software Dependency State

↓

Identify Configuration Baseline Governance Object

↓

Identify Software Object

↓

Establish Baseline Purpose

↓

Define Baseline Scope

↓

Determine Baseline Classification

↓

Identify Material Configuration Items

↓

Establish Configuration Item Boundaries

↓

Establish Configuration Item Relationships

↓

Establish Baseline Composition

↓

Identify Version and Artifact References

↓

Identify Parameter, Default, and Override References

↓

Establish Environmental Context

↓

Identify Environmental Assumptions and Constraints

↓

Identify Baseline Establishment Authority

↓

Assess Authority Scope

↓

Establish Baseline Approval Condition

↓

Determine Effective and Temporal Applicability

↓

Preserve Baseline Identity and Record Integrity

↓

Assess Baseline Completeness and Consistency

↓

Associate Baseline Evidence

↓

Assess Evidence Competence and Integrity

↓

Validate Material Baseline Conditions

↓

Identify Baseline Uncertainty

↓

Identify Baseline Conflicts

↓

Identify Baseline Gaps and Critical Gaps

↓

Determine Baseline Governance State

↓

Establish Governed Configuration Baseline State

↓

Preserve Revision, Supersession, and Continuity Relationships

↓

Handoff to Parent SCFS Governance

↓

Reassess When Material Baseline Conditions Change

The principal output is:

Governed Configuration Baseline State

Governance Outputs

CBM may produce:

Configuration Baseline Governance Object

Baseline Identity Record

Baseline Purpose Record

Baseline Scope Record

Baseline Classification Record

Configuration Item Register

Configuration Item Boundary Record

Configuration Relationship Map

Baseline Composition Record

Baseline Version Reference Register

Baseline Artifact Reference Register

Baseline Parameter Reference Register

Default Configuration Reference Record

Configuration Override Reference Record

Baseline Environmental Context Record

Baseline Environmental Assumption Record

Baseline Constraint Record

Baseline Establishment Authority Record

Baseline Authority Scope Record

Automated Baseline Establishment Record

Baseline Approval Record

Baseline Effective Condition Record

Baseline Temporal Applicability Record

Baseline Identification Integrity Record

Baseline Record Integrity Assessment

Baseline Completeness Assessment

Baseline Consistency Assessment

Baseline Validation Record

Baseline Evidence Register

Baseline Evidence Competence Assessment

Baseline Evidence Integrity Assessment

Baseline Uncertainty State

Baseline Conflict Record

Baseline Gap

Critical Baseline Gap

Baseline Change Reference

Baseline Revision Record

Baseline Supersession Record

Baseline Retirement Record

Baseline Reconstruction Record

Baseline Drift Reference

Baseline Continuity Record

Baseline Reassessment Record

Baseline Governance State

SCFS Baseline Handoff Record

The principal output is:

Governed Configuration Baseline State

Critical Distinctions

CBM preserves the following distinctions:

Documented Configuration ≠ Governed Configuration Baseline

Configuration Baseline Governance Object ≠ Configuration File

Baseline Purpose ≠ Universal Baseline Applicability

Defined Baseline Scope ≠ Complete Baseline Composition

Baseline Classification ≠ Baseline Governance State

Configuration Item Listing ≠ Verified Configuration Item Identity

Configuration Item Name ≠ Configuration Item Boundary

Configuration Item Presence ≠ Established Configuration Relationship

Configuration Item Collection ≠ Governed Baseline Composition

Version Label Presence ≠ Verified Version Correspondence

Artifact Reference ≠ Verified Artifact Identity

Baseline Parameter Value ≠ Effective Runtime Parameter Value

Implicit Default ≠ Governed Baseline Value

Baseline Reference ≠ Effective Configuration After Overrides

Test Baseline Validity ≠ Production Baseline Validity

Assumed Environment Conditions ≠ Verified Environment Conditions

Constraint Reference ≠ Verified Constraint Satisfaction

Ability to Establish a Baseline ≠ Authority to Establish a Baseline

Development Baseline Authority ≠ Production Baseline Authority

Automated Baseline Generation ≠ Authorized Baseline Establishment

Approved Baseline ≠ Technically Validated Baseline

Baseline Approval ≠ Immediate Baseline Applicability

Historical Baseline Validity ≠ Current Baseline Applicability

Baseline Filename ≠ Reliable Baseline Identity

Baseline Record Integrity ≠ Baseline Validity

Complete Baseline Record ≠ Complete Software Configuration Knowledge

Baseline Completeness ≠ Baseline Consistency

Validated Baseline ≠ Verified Actual Configuration

Baseline Evidence Presence ≠ Baseline Claim Validity

Evidence of Baseline Approval ≠ Evidence of Baseline Technical Validity

Intact Baseline Evidence ≠ Correct Baseline Composition

Unresolved Baseline Conditions ≠ Verified Baseline Conditions

Conflicting Baseline Records ≠ Permission to Select the Preferred Baseline

Baseline Evidence Gap ≠ Proven Baseline Failure

Critical Baseline Gap ≠ Automatic Software Failure

Baseline Change Recognition ≠ Configuration Change Authorization

Baseline Revision ≠ Silent Baseline Replacement

Baseline Supersession ≠ Historical Baseline Erasure

Retired Baseline ≠ Invalid Historical Baseline

Reconstructed Baseline ≠ Directly Observed Historical Configuration

Baseline Difference ≠ Automatically Unauthorized Configuration Drift

Baseline Continuity ≠ Baseline Immutability

Previously Valid Baseline ≠ Permanently Applicable Baseline

Established Baseline ≠ Verified Operational Configuration

Cross-Architecture Boundary

CBM governs software configuration baseline establishment and continuity

within SCFS and SLGA™.

It does not independently govern complete build execution, artifact-version integrity validation, configuration change execution, build reproducibility, software quality, software security, release authorization, or deployment authorization.

Within SCFS:

CBM — Configuration Baseline Module governs baseline identity,

composition, authority, applicability, evidence, and continuity.

BMM — Build Management Module governs build inputs, execution conditions,

and build-to-artifact correspondence.

AVIM — Artifact Version Integrity Module governs artifact identity,

version correspondence, and integrity.

CCCM — Configuration Change Control Module governs configuration change

authority, execution, and resulting-state relationships.

BREM — Build Reproducibility & Environment Module governs build

environment identity, reproducibility conditions, and reconstruction evidence.

Within SLGA™:

Software Requirements Standard (SRS) governs software requirements.

Software Creation Standard (SCS) governs software creation.

Software Dependency Standard (SDS) governs software dependencies.

Software Quality Standard (SQS) governs software quality.

Software Security Standard (SSS) governs software security.

Software Release Standard (SRLS) governs software release.

Software Deployment Standard (SDPS) governs software deployment.

Software Change Management Standard (SCMS) governs broader lifecycle

change management.

Software Retirement Standard (SRTS) governs software retirement.

Software Lifecycle Control Standard (SLCS) governs cross-cutting

lifecycle control.

Relevant interoperating OOF® architectures include:

GOA™ — Governance Architecture: General authority and delegation

governance.

AGA™ — Accountability Governance Architecture: Relevant responsibility

and accountability relationships.

ORA™ — Operational Reality Architecture: Competent assessment of actual

operational conditions.

ASGA™ — Autonomous Systems Governance Architecture: Relevant autonomous

baseline establishment and operational boundaries.

AGI™ — Agent Governance Infrastructure: Relevant agent permissions and

tool-path governance.

CBM preserves configuration-baseline-specific governance without replacing

these competent domains.

Canonical Boundary

CBM begins when a material software configuration requires recognition as an identifiable baseline reference for subsequent configuration or lifecycle governance.

CBM governs:

• Baseline governance objects

• Baseline identity

• Baseline purpose

• Baseline scope

• Baseline classification

• Configuration items

• Configuration item boundaries

• Configuration item relationships

• Baseline composition

• Version references

• Artifact references

• Parameter references

• Default configuration references

• Override references

• Environmental context

• Environmental assumptions

• Baseline constraints

• Establishment authority

• Authority scope

• Automated baseline establishment

• Baseline approval

• Effective conditions

• Temporal applicability

• Identification integrity

• Record integrity

• Baseline completeness

• Baseline consistency

• Baseline validation

• Baseline evidence

• Evidence competence

• Evidence integrity

• Baseline uncertainty

• Baseline conflicts

• Baseline gaps

• Critical baseline gaps

• Baseline change references

• Baseline revisions

• Baseline supersession

• Baseline retirement

• Baseline reconstruction

• Baseline drift references

• Baseline continuity

• Baseline reassessment

• Baseline governance states

CBM does not independently determine:

• Complete software configuration validity

• Actual runtime configuration conformity

• Build execution correctness

• Build reproducibility

• Artifact-version integrity

• Configuration change authorization outside baseline establishment

• Software quality

• Software security

• Release authorization

• Deployment authorization

• Legal compliance

• Legal liability

CBM ends when the relevant baseline conditions have been represented as a:

Governed Configuration Baseline State

with sufficient identity, purpose, scope, composition, authority,

applicability, evidence, validation, uncertainty, and continuity information for competent parent SCFS governance.

Use Case 1 — An AI Coding Agent Establishes a New Baseline Without

Production Authority

An organization uses an AI coding agent to maintain a software application.

The agent receives authorization to modify application code and prepare a

development build.

During execution, the agent generates a new configuration baseline that includes:

• Updated component versions

• Modified configuration parameters

• Changed service references

• New environment assumptions

The agent labels the new baseline as the current production baseline.

The baseline is technically coherent.

The configuration files are complete.

However, the agent was authorized only to prepare a development

configuration.

The agent did not possess authority to establish a new production baseline.

CBM evaluates:

Baseline Identity → Purpose → Scope → Classification → Composition →

Authority → Approval → Effective Conditions

The assessment identifies:

• A new baseline object was created

• The baseline had an identifiable composition

• The agent's authority was limited to development activity

• Production baseline establishment required separate competent

• The baseline label did not establish legitimate production

authority. applicability.

The relevant governance state may be:

Authority-Unresolved

or, where lack of authority is established:

Invalid

The critical distinctions are:

Automated Baseline Generation ≠ Authorized Baseline Establishment.

Development Baseline Authority ≠ Production Baseline Authority.

Baseline Classification ≠ Baseline Governance State.

The result: CBM prevents a technically coherent configuration reference

from becoming an authoritative production baseline solely because an automated system created or labeled it.

Use Case 2 — A Production Baseline Cannot Be Reconstructed After a

Critical Failure

An organization operates a software service that has undergone several years of maintenance.

The service has an approved production configuration baseline.

However, the baseline record contains only:

• An application version label

• A partial configuration file

• A general environment description

The baseline does not sufficiently identify:

• The exact runtime version

• Several material component versions

• Relevant configuration defaults

• Required environment-specific parameters

• The effective configuration of an external integration

A critical failure occurs.

The organization attempts to reconstruct the previous production

configuration.

The recorded baseline is insufficient to establish the material configuration reference required for competent restoration.

CBM evaluates:

Baseline Identity → Scope → Configuration Items → Version References →

Parameter References → Environmental Context → Completeness → Reconstruction

The assessment identifies:

• The baseline had an identifiable label

• The baseline composition was materially incomplete

• Several critical configuration relationships were unresolved

• The available evidence was insufficient to reconstruct the intended

• Historical approval did not establish sufficient baseline completeness

reference state.

The relevant governance state may be:

Composition-Unresolved

or:

Evidence-Limited

Where the missing information prevents competent assessment of a critical configuration reference, CBM may establish a:

Critical Baseline Gap

The critical distinctions are:

Baseline Filename ≠ Reliable Baseline Identity.

Complete Baseline Record ≠ Complete Software Configuration Knowledge.

Approved Baseline ≠ Technically Validated Baseline.

Reconstructed Baseline ≠ Directly Observed Historical Configuration.

The result: CBM reveals that a baseline is operationally useful only when its material composition, context, authority, and evidence remain sufficiently established to support competent interpretation and reconstruction.

Canonical Closing Statement

Software configuration cannot remain governable without a reliable

reference state.

A software organization may possess configuration files, repository commits, build manifests, and version labels.

Yet these records do not automatically establish which configuration was

legitimately adopted, which components and parameters it contained, which environment it applied to, or whether its material relationships can be reconstructed.

A baseline is more than a saved configuration.

It is an identifiable governance reference.

Its authority must be established.

Its scope must be defined.

Its composition must be sufficiently known.

Its evidence must support the claims made about it.

Its applicability must remain bounded.

And its historical relationships must survive subsequent changes.

Documented Configuration ≠ Governed Configuration Baseline.

Approved Baseline ≠ Technically Validated Baseline.

Baseline Record Integrity ≠ Baseline Validity.

Validated Baseline ≠ Verified Actual Configuration.

Baseline Continuity ≠ Baseline Immutability.

These distinctions become increasingly important as software configuration

is generated, revised, and maintained through automated systems and autonomous development mechanisms.

An AI agent may generate a technically coherent baseline.

A pipeline may record it.

A repository may preserve it.

A deployment system may reference it.

None of these actions independently establishes legitimate baseline

authority or proves that the resulting operational configuration conforms to that reference.

CBM establishes the methodological requirement that configuration baselines remain identifiable, authorized, evidence-supported, and reconstructable reference states.

It does not require every baseline to remain unchanged.

It requires changes, revisions, supersession, and retirement to preserve

material baseline identity and historical continuity.

CBM transforms software configuration baselines from technical snapshots

into governed reference states — ensuring that software systems can establish not merely which configuration was recorded, but which configuration reference was legitimately established, what it contained, where it applied, and whether its identity and evidence remain sufficient for subsequent lifecycle governance.

A configuration baseline is not governed because it has been saved, versioned, or approved. It is governed when its purpose, scope, composition, authority, applicability, evidence, and continuity remain sufficiently established to serve as a reliable reference for the software configuration states that follow.